Software Engineering Lab 7

Fuzz Testing for Input Validation Bugs

Name: Ayaan Ahmed

SRN: PES1UG24CS102

Section: B

Date: 11 November 2026

Application: Laundry Service

Repository: https://github.com/ayaan193/SE-labs/tree/main/LAB7

The instructor provides a separate Lab 7 starter for the laundry-service topic used in Lab 6. Its three functions are parse_order, weight_average and payable_charge. The original starter is retained as processor_original.py; the corrected submission is processor.py.

Fuzz testing found unhandled exceptions on malformed strings, empty strings and None. The original implementation also truncates the average of 3,4 to 3. The corrected application validates types, separators and numeric inputs; it rejects invalid data without crashing and returns an arithmetic mean of 3.5.

Validation policy: accept regular, delicate and bedding garments; require finite nonnegative numeric values; return None for invalid input and display Invalid input in the interactive application. Payable charge retains 18% tax and a zero lower bound.

Results: all three original no-crash tests fail. All 23 final test cases pass. Six Hypothesis tests generate inputs; the three no-crash tests use 300 examples each and the valid-input tests use 100 examples each.

This single PDF includes all 11 supplied Lab 7 screenshots followed by complete listings of processor.py and test_processor.py. Earlier captures include optional pytest cache warnings; the final capture shows 23 passed with the cache provider disabled.

Original application execution

Inputs regular:6, 3,4 and 300|40. The original average is incorrectly truncated to 3.

```
PS C:\WINDOWS\system32> cd "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7"
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> .\.venv\Scripts\python.exe processor_original.py
==== Running parse_order ====
Enter garment:weight in kg (e.g. regular:6): regular:6
Garment and Weight: ('Regular', 6.0)

==== Running weight_average ====
Enter garment weights as CSV (e.g. 3,5,8): 3,4
Average Weight: 3

==== Running payable_charge ====
Enter charge|discount (e.g. 300|40): 300|40
Payable Charge: 314.0
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7>
```

Original application: fuzz tests fail

Hypothesis-based no-crash tests fail for all three original functions.

```
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> $env:PROCESSOR_MODULE = "processor_original"
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> .\.venv\Scripts\python.exe -m pytest test_processor.py -k no_cras
h -v --tb=short
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\.venv
\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase(WindowsPath('C:/Users/Zia Ahmed/Downloads/3rd_Yea
r/SE-labs/LAB7/.hypothesis/examples'))
rootdir: C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7
plugins: hypothesis-6.112.1
collected 23 items / 20 deselected / 3 selected

test_processor.py::test_parse_order_no_crash FAILED                                                  [ 33%]
test_processor.py::test_weight_average_no_crash FAILED                                               [ 66%]
test_processor.py::test_payable_charge_no_crash FAILED                                               [100%]
```

Original failure summary

Three tests fail; 20 are deselected. The cache warnings concern pytest storage permissions.

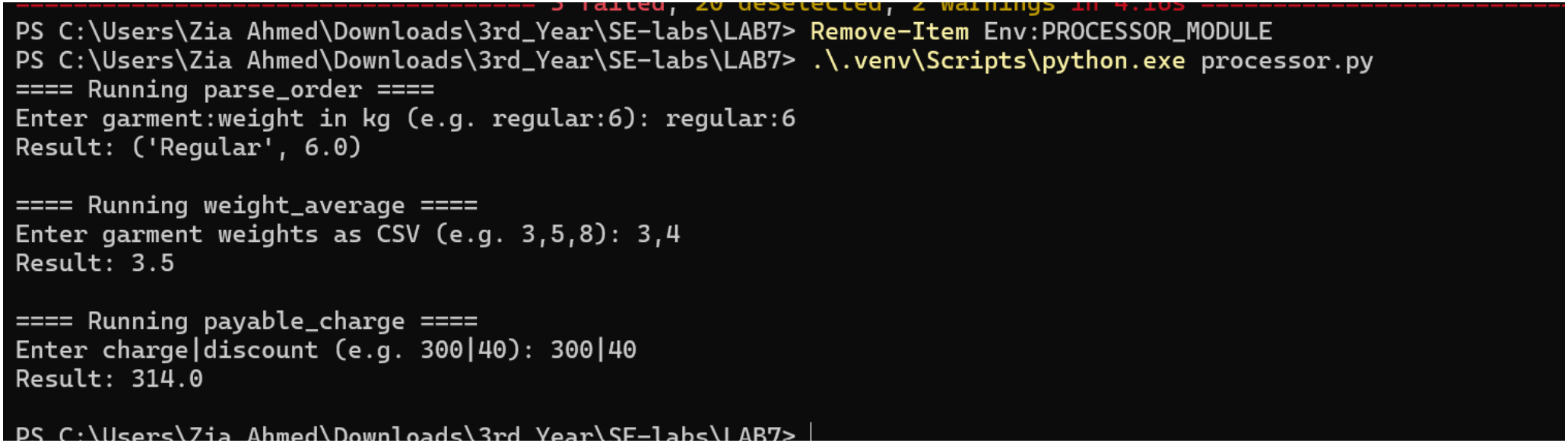
```
============================================== warnings summary ===============================================
.venv\Lib\site-packages\_pytest\cacheprovider.py:487
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\.venv\Lib\site-packages\_pytest\cacheprovider.py:487: PytestCacheWa
rning: could not create cache path C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\.pytest_cache\v\cache\nodeids: [Wi
nError 5] Access is denied: 'C:\\Users\\Zia Ahmed\\Downloads\\3rd_Year\\SE-labs\\LAB7\\.pytest_cache\\v\\cache'
    config.cache.set("cache/nodeids", sorted(str(n) for n in self.cached_nodeids))

.venv\Lib\site-packages\_pytest\cacheprovider.py:439
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\.venv\Lib\site-packages\_pytest\cacheprovider.py:439: PytestCacheWa
rning: could not create cache path C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\.pytest_cache\v\cache\lastfailed:
[WinError 5] Access is denied: 'C:\\Users\\Zia Ahmed\\Downloads\\3rd_Year\\SE-labs\\LAB7\\.pytest_cache\\v\\cache'
    config.cache.set("cache/lastfailed", current_lastfailed)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================ short test summary info ============================================
FAILED test_processor.py::test_parse_order_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-excep
tions)
FAILED test_processor.py::test_weight_average_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-ex
ceptions)
FAILED test_processor.py::test_payable_charge_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-ex
ceptions)
================================== 3 failed, 20 deselected, 2 warnings in 4.16s ===================================
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7>
```

Corrected application execution

The same inputs now produce average 3.5; the payable charge remains 314.0.



Corrected application: passing tests

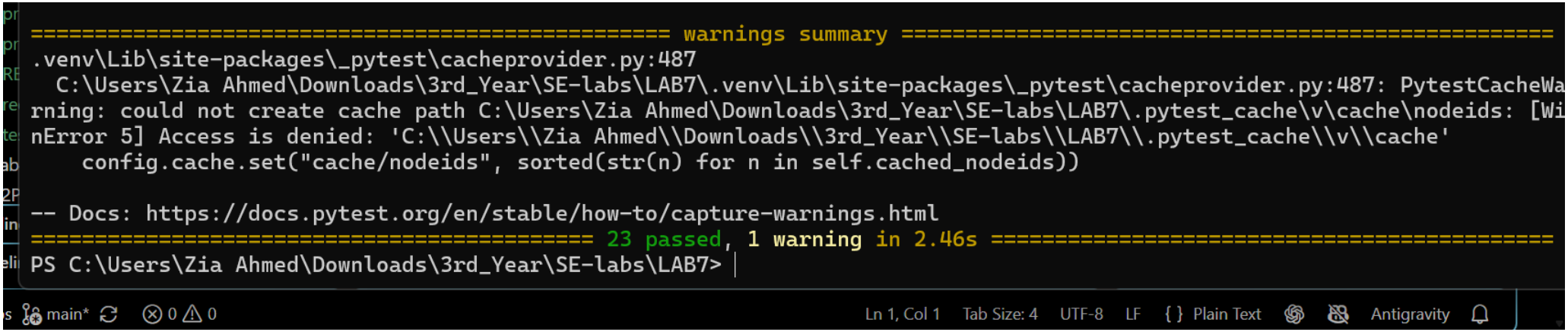
The no-crash properties and valid-input properties pass against the corrected module.

```
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> .\.venv\Scripts\python.exe -m pytest test_processor.py -v
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\.venv
\Scripts\python.exe
cachedir: .pytest_cache
hypothesis profile 'default' -> database=DirectoryBasedExampleDatabase(WindowsPath('C:/Users/Zia Ahmed/Downloads/3rd_Yea
r/SE-labs/LAB7/.hypothesis/examples'))
rootdir: C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7
plugins: hypothesis-6.112.1
collected 23 items

test_processor.py::test_parse_order_no_crash PASSED                                                    [  4%]
test_processor.py::test_weight_average_no_crash PASSED                                                 [  8%]
test_processor.py::test_payable_charge_no_crash PASSED                                                 [ 13%]
test_processor.py::test_parse_order_valid PASSED                                                       [ 17%]
test_processor.py::test_weight_average_valid PASSED                                                    [ 21%]
test_processor.py::test_payable_charge_valid PASSED                                                    [ 26%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-None] PASSED                           [ 30%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-] PASSED                               [ 34%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-regular:-1] PASSED                     [ 39%]
```

Initial successful execution summary

All 23 tests pass. This earlier run includes an optional pytest cache warning.



Corrected application: invalid inputs

Negative weight, an empty CSV field and a NaN discount are safely rejected.

```
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7> .\.venv\Scripts\python.exe processor.py
==== Running parse_order ====
Enter garment:weight in kg (e.g. regular:6): regular:-1
Invalid input

==== Running weight_average ====
Enter garment weights as CSV (e.g. 3,5,8): 3,,4
Invalid input

==== Running payable_charge ====
Enter charge|discount (e.g. 300|40): 300|nan
Invalid input

PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7>
```

Fuzz failure details: parse_order

Counterexamples include a colon without a numeric weight and an empty string.

```
=================================================== FAILURES ===================================================
_____________________________________________ test_parse_order_no_crash _____________________________________________
  + Exception Group Traceback (most recent call last):
  |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 21, in test_parse_order_no_crash
  |     @given(arbitrary_input)
  |                    ^^^
  | ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 23, in test_parse_order_no_crash
    |     result = app.parse_order(value)
    |              ^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\processor_original.py", line 3, in parse_order
    |     return garment.strip().title(), float(kg)
    |                                     ^^^^^^^^^
    | ValueError: could not convert string to float: ''
    | Falsifying example: test_parse_order_no_crash(
    |     value=':',
    | )
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 23, in test_parse_order_no_crash
    |     result = app.parse_order(value)
    |              ^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\processor_original.py", line 2, in parse_order
    |     garment, kg = text.split(':')
    |     ^^^^^^^^^^^
    | ValueError: not enough values to unpack (expected 2, got 1)
    | Falsifying example: test_parse_order_no_crash(
    |     value='',
    | )
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 23, in test_parse_order_no_crash
    |     result = app.parse_order(value)
    |              ^^^^^^^^^^^^^^^^^^^^^^
    |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\processor_original.py", line 2, in parse_order
    |     garment, kg = text.split(':')
    |                   ^^^^^^^^^^
    | AttributeError: 'NoneType' object has no attribute 'split'
```

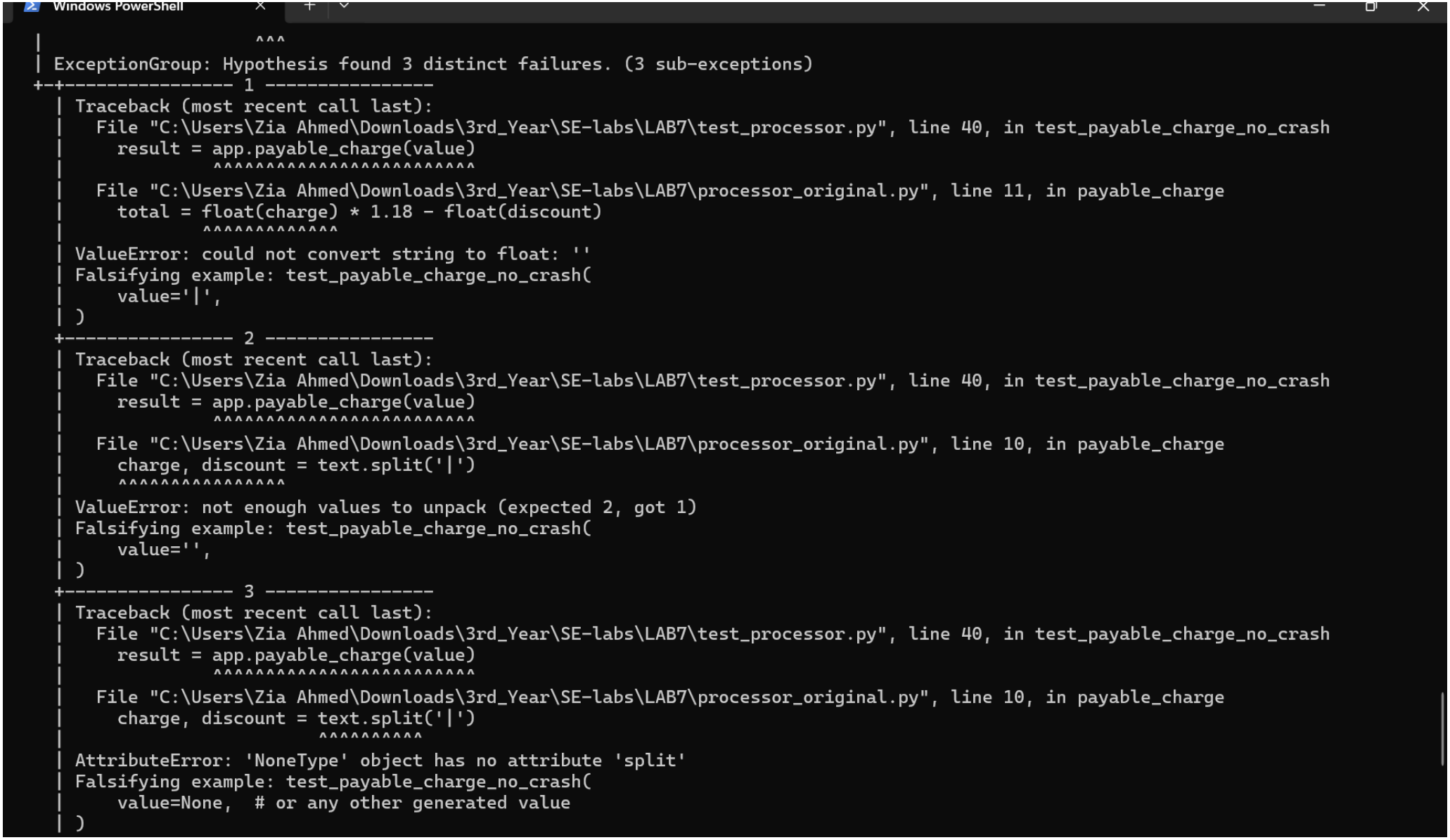
Fuzz failure details: weight_average

An empty string causes ValueError; None causes AttributeError in the original module.

```
  | Falsifying example: test_parse_order_no_crash(
  |     value=None,  # or any other generated value
  | )
  +------------------------------------
_____________________________________ test_weight_average_no_crash ______________________________________
+ Exception Group Traceback (most recent call last):
|   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 31, in test_weight_average_no_crash
|     @given(arbitrary_input)
|                 ^^^
| ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
+-+---------------- 1 ----------------
  | Traceback (most recent call last):
  |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 33, in test_weight_average_no_crash
  |     result = app.weight_average(value)
  |              ^^^^^^^^^^^^^^^^^^^^^^^^^
  |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\processor_original.py", line 6, in weight_average
  |     values = [int(r) for r in weights_csv.split(',')]
  |               ^^^^^^
  | ValueError: invalid literal for int() with base 10: ''
  | Falsifying example: test_weight_average_no_crash(
  |     value='',
  | )
  +---------------- 2 ----------------
  | Traceback (most recent call last):
  |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 33, in test_weight_average_no_crash
  |     result = app.weight_average(value)
  |              ^^^^^^^^^^^^^^^^^^^^^^^^^
  |   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\processor_original.py", line 6, in weight_average
  |     values = [int(r) for r in weights_csv.split(',')]
  |                              ^^^^^^^^^^^^^^^^
  | AttributeError: 'NoneType' object has no attribute 'split'
  | Falsifying example: test_weight_average_no_crash(
  |     value=None,  # or any other generated value
  | )
  +------------------------------------
_____________________________________ test_payable_charge_no_crash ______________________________________
+ Exception Group Traceback (most recent call last):
|   File "C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7\test_processor.py", line 38, in test_payable_charge_no_crash
|     @given(arbitrary_input)
|                 ^^^
```

Fuzz failure details: payable_charge

Malformed separators, empty strings and None expose missing validation in the original module.



Final test execution: 23 passed

Clean final execution after disabling the optional pytest cache provider.

```
test_processor.py::test_parse_order_no_crash PASSED                                  [  4%]
test_processor.py::test_weight_average_no_crash PASSED                               [  8%]
test_processor.py::test_payable_charge_no_crash PASSED                               [ 13%]
test_processor.py::test_parse_order_valid PASSED                                     [ 17%]
test_processor.py::test_weight_average_valid PASSED                                  [ 21%]
test_processor.py::test_payable_charge_valid PASSED                                  [ 26%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-None] PASSED         [ 30%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-] PASSED             [ 34%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-regular:-1] PASSED   [ 39%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-regular:nan] PASSED  [ 43%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-regular:inf] PASSED  [ 47%]
test_processor.py::test_invalid_inputs_are_rejected[parse_order-unknown:5] PASSED    [ 52%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-None] PASSED      [ 56%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-] PASSED          [ 60%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-3,,5] PASSED      [ 65%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average--1,5] PASSED      [ 69%]
test_processor.py::test_invalid_inputs_are_rejected[weight_average-nan,5] PASSED     [ 73%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-None] PASSED      [ 78%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-] PASSED          [ 82%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge--10|2] PASSED     [ 86%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-300|nan] PASSED   [ 91%]
test_processor.py::test_invalid_inputs_are_rejected[payable_charge-1e309|0] PASSED   [ 95%]
test_processor.py::test_fractional_average_regression PASSED                        [100%]

============================================== 23 passed in 2.90s ==============================================
PS C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB7>
```

processor.py - complete source code

Lines 1-60 of 70

```
"""Lab 7 laundry input processing. Invalid input returns None."""

import math

GARMENTS = {"regular", "delicate", "bedding"}


def _nonnegative_number(text):
    """Parse a finite, nonnegative number; return None on invalid input."""
    try:
        value = float(text)
    except (ValueError, TypeError, OverflowError):
        return None
    return value if math.isfinite(value) and value >= 0 else None


def parse_order(text):
    if not isinstance(text, str) or text.count(":") != 1:
        return None
    garment, raw_weight = text.split(":")
    garment = garment.strip().lower()
    weight = _nonnegative_number(raw_weight)
    if garment not in GARMENTS or weight is None:
        return None
    return garment.title(), weight


def weight_average(weights_csv):
    if not isinstance(weights_csv, str) or not weights_csv.strip():
        return None
    values = [_nonnegative_number(part) for part in weights_csv.split(",")]
    if any(value is None for value in values):
        return None
    # Divide first so even a list of very large finite weights can be averaged.
    try:
        average = math.fsum(value / len(values) for value in values)
    except OverflowError:
        return None
    return average if math.isfinite(average) else None


def payable_charge(text):
    if not isinstance(text, str) or text.count("|") != 1:
        return None
    raw_charge, raw_discount = text.split("|")
    charge = _nonnegative_number(raw_charge)
    discount = _nonnegative_number(raw_discount)
    if charge is None or discount is None:
        return None
    total = charge * 1.18 - discount
    if not math.isfinite(total):
        return None
    return max(total, 0.0)


def main():
    for heading, prompt, function in [
        ("parse_order", "Enter garment:weight in kg (e.g. regular:6): ", parse_order),
        ("weight_average", "Enter garment weights as CSV (e.g. 3,5,8): ", weight_average),
        ("payable_charge", "Enter charge|discount (e.g. 300|40): ", payable_charge),
```

processor.py (continued)

Lines 61-70 of 70

```
    ]:
        print(f"==== Running {heading} ====")
        result = function(input(prompt))
        print("Invalid input" if result is None else f"Result: {result}")
        print()


if __name__ == "__main__":
    main()

```

test_processor.py - complete source code

Lines 1-60 of 91

```
"""Use PROCESSOR_MODULE=processor_original to reproduce original failures."""

import importlib
import math
import os

import pytest
from hypothesis import given, settings, strategies as st

app = importlib.import_module(os.environ.get("PROCESSOR_MODULE", "processor"))

arbitrary_input = st.one_of(
    st.none(), st.text(), st.booleans(), st.integers(), st.floats(),
    st.lists(st.integers(), max_size=5),
    st.dictionaries(st.text(max_size=10), st.integers(), max_size=3),
)
weights = st.floats(min_value=0, max_value=1_000_000, allow_nan=False, allow_infinity=False)


@settings(max_examples=300)
@given(arbitrary_input)
def test_parse_order_no_crash(value):
    result = app.parse_order(value)
    if result is not None:
        garment, weight = result
        assert garment in {"Regular", "Delicate", "Bedding"}
        assert math.isfinite(weight) and weight >= 0


@settings(max_examples=300)
@given(arbitrary_input)
def test_weight_average_no_crash(value):
    result = app.weight_average(value)
    assert result is None or (math.isfinite(result) and result >= 0)


@settings(max_examples=300)
@given(arbitrary_input)
def test_payable_charge_no_crash(value):
    result = app.payable_charge(value)
    assert result is None or (math.isfinite(result) and result >= 0)


@given(st.sampled_from(["regular", "delicate", "bedding"]), weights)
def test_parse_order_valid(garment, weight):
    assert app.parse_order(f" {garment.upper()} : {weight} ") == (garment.title(), weight)


@given(st.lists(weights, min_size=1, max_size=30))
def test_weight_average_valid(values):
    result = app.weight_average(",".join(map(str, values)))
    assert result == pytest.approx(sum(values) / len(values))
    assert min(values) <= result + 1e-8
    assert result <= max(values) + 1e-8


@given(weights, weights)
def test_payable_charge_valid(charge, discount):
    assert app.payable_charge(f"{charge}|{discount}") == pytest.approx(
        max(charge * 1.18 - discount, 0)
```

test_processor.py (continued)

Lines 61-91 of 91

```
    )


@pytest.mark.parametrize(
    "function,value",
    [
        ("parse_order", None),
        ("parse_order", ""),
        ("parse_order", "regular:-1"),
        ("parse_order", "regular:nan"),
        ("parse_order", "regular:inf"),
        ("parse_order", "unknown:5"),
        ("weight_average", None),
        ("weight_average", ""),
        ("weight_average", "3,,5"),
        ("weight_average", "-1,5"),
        ("weight_average", "nan,5"),
        ("payable_charge", None),
        ("payable_charge", ""),
        ("payable_charge", "-10|2"),
        ("payable_charge", "300|nan"),
        ("payable_charge", "1e309|0"),
    ],
)
def test_invalid_inputs_are_rejected(function, value):
    assert getattr(app, function)(value) is None


def test_fractional_average_regression():
    assert app.weight_average("3,4") == 3.5

```